Feature Selection in Statistical Machine Learning

Garvesh Raskutti

Lecture 12

Recap and outline

- Validating feature selection methods
- A deeper look at stability
- Stability selection

Feature selection validation approaches

- Out-of-sample prediction error
  - Default and quantitative
  - May over-select features
- Domain knowledge
  - Often subjective and not always straightforward to find
  - Can be biased by domain knowledge
- Stability
  - We can make it quantitative
  - Main focus of today
- Simplicity/interpretability
  - Again subjective and context-dependent

What is stability?

What is stability?

- If *some aspect of pipeline* changes slightly how much does output change?
- The *some aspect of pipeline* is extremely vague and gives us many options
- Often most common: data perturbations (if a small number of data points changes or is included/excluded, how does your outcome change)?
- Mathematically, let D and D' be datasets that are *close* by some metric $d(.,.)$ (e.g. replace or remove a single sample) and let $S(.)$ be a learned subset based on data,

$$\text{Stability}(S(.)) = \sup_{d(D,D')<\epsilon} |S(D) - S(D')|$$

Stability in feature selection

- Goal of feature selection to a select a subset $S \subset \{X_1, X_2, ..., X_p\}$ that influence prediction Y
- Mathematically, let D and D' be datasets that are *close* by some metric $d(.,.)$ (e.g. replace or remove a single sample) and let $S(.)$ be a learned subset based on data,

$$\text{Stability}(S(.)) = \sup_{d(D,D')<\epsilon} |S(D) - S(D')|$$

- How close are the two subsets $S(D)$ to $S(D')$?
- Or alternatively, does feature X_k appear in both $S(D)$ and $S(D')$ (stability selection)?
- Often need to consider multiple data perturbations

How high correlation impacts stability intuitively

- Correlated/dependent features typically lead to unstable feature selection
- Methods such as Lasso tend to pick either correlated feature
- Feature selection becomes close to un-identifiable
- Stability is one way of dealing with this

Stability example

$$\begin{aligned} Y &= X_1 + w \\ X_2 &= X_1 + \delta \end{aligned}$$

where $w \sim \text{Normal}(0, \tau^2)$, $\delta \sim \text{Normal}(0, \sigma^2)$, $X_1 \sim \text{Normal}(0, 1)$ are independent

- Lasso or other standard feature selection methods likely to pick either feature particularly if σ^2 is small
- So if data is perturbed slightly, likely to switch between selecting X_1 and X_2
- Later we will refer back to this example to see how stability selection works

One approach: How stable are different methods

- How stable is a method under different data perturbations
- For feature selection, we can see how stable subsets are under different data perturbations (often slow)
- Many different metrics to compare subset similarity
- One of the most common is Jaccard index

Jaccard index:

$$J(S_1, S_2) = \frac{|S_1 \cap S_2|}{|S_1 \cup S_2|}$$

It follows that $0 \leq J(S_1, S_2) \leq 1$ where $J(S_1, S_2) = 0$ if $S_1 \cap S_2 = \emptyset$ and $J(S_1, S_2) = 1$ if $S_1 = S_2$.

Bootstrap stability

- Bootstrap is a principle that involves sub-sampling/re-sampling from full dataset and seeing how value changes
- Connected to stability/variance
- Used for random forests and permutation testing
- We can also use it for computing stability
- Model-agnostic but computationally intensive

Bootstrap stability - code

```
def jaccard(set1, set2):
    return len(set1 & set2) / len(set1 | set2)

def bootstrap_stability(method_func, X, y, B=50):
    selections = []
    for _ in range(B):
        Xb, yb = resample(X, y)
        selections.append(set(method_func(Xb, yb)))

    scores = []
    for i in range(len(selections)):
        for j in range(i+1, len(selections)):
            scores.append(jaccard(selections[i], selections[j]))
    return np.mean(scores)

stability = {}

stability["AIC"] = bootstrap_stability(
    lambda Xb, yb: forward_selection(Xb, yb, "aic"),
    X_train, y_train
)

stability["BIC"] = bootstrap_stability(
    lambda Xb, yb: forward_selection(Xb, yb, "bic"),
    X_train, y_train
)

stability["Lasso"] = bootstrap_stability(
    lambda Xb, yb: list(X.columns[
        LassoCV(cv=5).fit(StandardScaler().fit_transform(Xb), yb).coef_ != 0
    ]),
    X_train, y_train
)
```

Stability results

- For Hitters data, we compared stability for three methods, AIC, BIC and Lasso
- For AIC: 0.40, BIC: 0.30, Lasso: 0.58
- Lasso most stable but no methods particularly stable
- Highly correlated features lead to instability

Conclusions of that approach

- Gives metric for a method
- Does not lead to a method to select features
- Useful concept: stability selection

Stability selection: Idea

- Run method multiple times over different splits/perturbations of the data
- Determine a feature set for each split of the data
- Assign a probability to a feature being selected by averaging over whether it appears in each split

Example: Adult income dataset

- Predicting whether adult income above $50k$ (UCI repository)
- From 1994 US census data
- $n \approx 48k$
- About 15 – 20 raw features but with many categories

Pre-processing adult income dataset

```
:
import pandas as pd
import numpy as np

# Column names
columns = [
    "age", "workclass", "fnlwgt", "education", "education_num",
    "marital_status", "occupation", "relationship", "race",
    "sex", "capital_gain", "capital_loss", "hours_per_week",
    "native_country", "income"
]

df = pd.read_csv(
    "adult.data",
    names=columns,
    sep=",",
    skipinitialspace=True,
    na_values="?"
)

df = df.dropna()

# Binary target
df["income"] = (df["income"] == ">50K").astype(int)

# One-hot encode
df_encoded = pd.get_dummies(df, drop_first=True)

X = df_encoded.drop("income", axis=1)
y = df_encoded["income"]
```

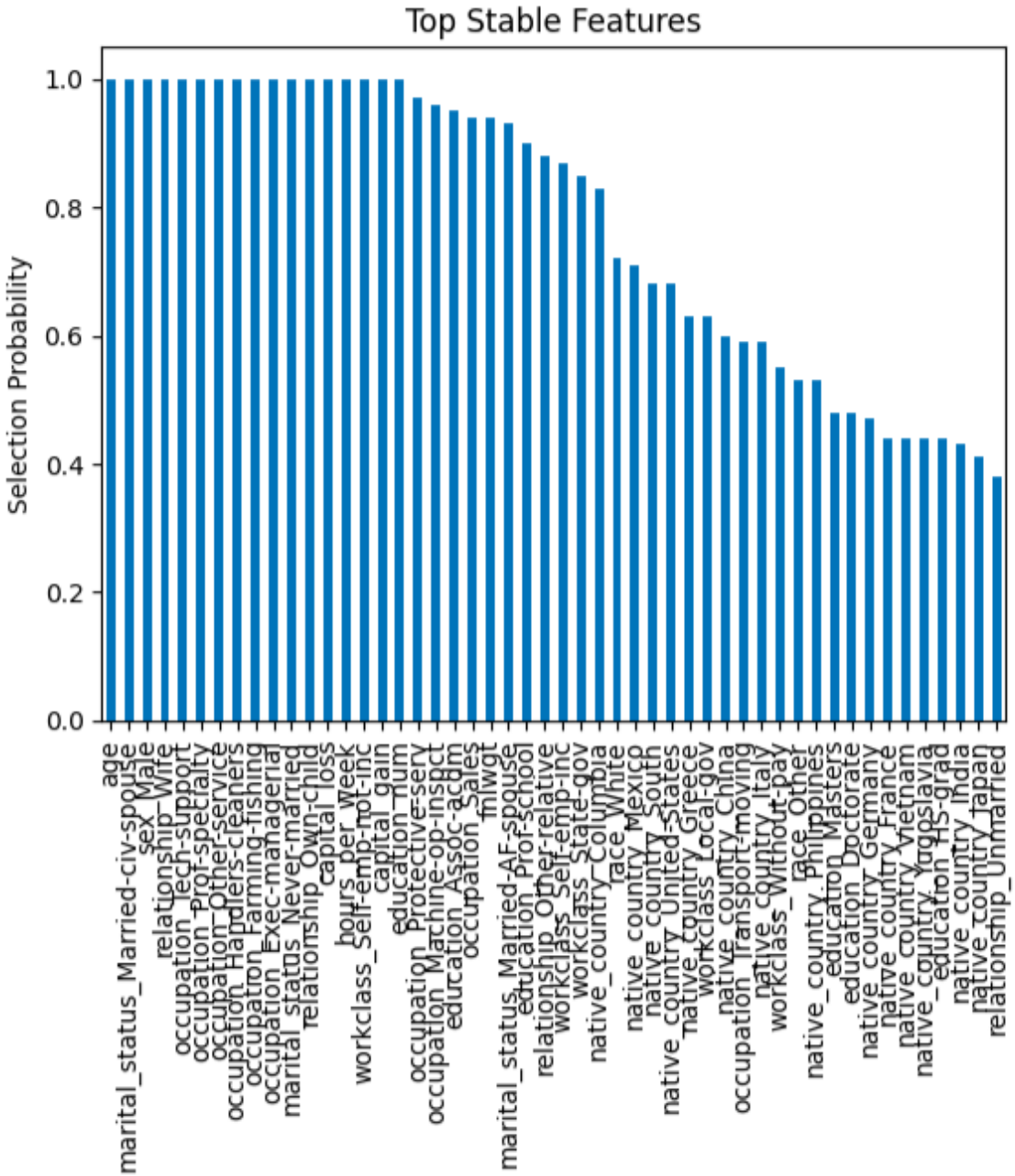
Stability selection code

```python
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```python
from sklearn.linear_model import LogisticRegression
from sklearn.utils import resample

n_iterations = 100
n_samples = len(y) // 2
selection_counts = np.zeros(X.shape[1])

C_value = 0.02  # corresponds to lambda = 1/C

for i in range(n_iterations):
    X_sub, y_sub = resample(X_scaled, y, n_samples=n_samples)

    model = LogisticRegression(
        penalty="l1",
        solver="liblinear",
        C=C_value,
        max_iter=1000
    )

    model.fit(X_sub, y_sub)

    selection_counts += (model.coef_[0] != 0)

selection_prob = selection_counts / n_iterations
```

```python
feature_stability = pd.Series(selection_prob, index=X.columns)
feature_stability = feature_stability.sort_values(ascending=False)

print(feature_stability)
```

Stability selection results



How do we select features?

- Stability selection returns a probability of selecting each feature
- Need to determine threshold probability π_{thr} where we select features with probability above this threshold
- How do we select π_{thr}? Often quite subjective

Selected subset for adult income data: $\pi_{\text{thr}} = 1$

```
feature_stability[feature_stability >= 1]
```

```
age                                1.0
marital_status_Married-civ-spouse  1.0
sex_Male                           1.0
relationship_Wife                  1.0
occupation_Tech-support            1.0
occupation_Prof-specialty          1.0
occupation_Other-service           1.0
occupation_Handlers-cleaners       1.0
occupation_Farming-fishing         1.0
occupation_Exec-managerial         1.0
marital_status_Never-married       1.0
relationship_Own-child             1.0
capital_loss                       1.0
hours_per_week                     1.0
education_num                      1.0
workclass_Self-emp-not-inc         1.0
capital_gain                       1.0
```

Selected subset for adult income data: $\pi_{\text{thr}} = 0.8$

```
feature_stability[feature_stability >= 0.8]
```

```
age                                1.00
marital_status_Married-civ-spouse  1.00
sex_Male                           1.00
relationship_Wife                  1.00
occupation_Tech-support            1.00
occupation_Prof-specialty          1.00
occupation_Other-service           1.00
occupation_Handlers-cleaners       1.00
occupation_Farming-fishing         1.00
occupation_Exec-managerial         1.00
marital_status_Never-married       1.00
relationship_Own-child             1.00
capital_loss                       1.00
hours_per_week                     1.00
education_num                      1.00
workclass_Self-emp-not-inc         1.00
capital_gain                       1.00
occupation_Protective-serv         0.98
fnlwgt                             0.96
occupation_Machine-op-inspct       0.95
occupation_Sales                   0.92
education_Prof-school              0.90
marital_status_Married-AF-spouse   0.89
relationship_Other-relative        0.89
workclass_Self-emp-inc             0.89
native_country_Columbia            0.89
workclass_State-gov                0.88
education_Assoc-acdm               0.86
```

False positive and false negative

- Need to balance between potential *false positives* and *false negatives*
- False positives means selecting a feature that is not in the original set
- False negatives mean not selecting a feature that is in the true feature set
- Need knowledge of true feature set which you can know for simulated but not for real data (unless there is a priori knowledge)

Useful measures:

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{F1} = \frac{2TP}{2TP + FP + FN}$$

Relating selection probability to false positive rate

Derived in Meinshausen and Buhlmann '08:

$$\mathbb{E}[V] \leq \frac{q^2}{(2\pi_{\text{thr}} - 1)p}$$

where

- V is number of false positives
- q expected number of selections per sub-sample
- p is total number of features

Simulation example

$$\begin{aligned} Y &= X_1 + w \\ X_2 &= X_1 + \delta \end{aligned}$$

where $w \sim \text{Normal}(0, \tau^2)$, $\delta \sim \text{Normal}(0, \sigma^2)$, $X_1 \sim \text{Normal}(0, 1)$ are independent

Simulation example - Preparation code

```python
import numpy as np
import pandas as pd
from sklearn.linear_model import Lasso
from sklearn.preprocessing import StandardScaler

# ----------------------------
# 1. Generate Data
# ----------------------------
def generate_data(n=200, p_noise=1, sigma_delta=0.3, sigma_eps=1.0, seed=None):
    rng = np.random.RandomState(seed)

    X1 = rng.normal(size=n)
    delta = rng.normal(scale=sigma_delta, size=n)
    X2 = X1 + delta

    noise = rng.normal(size=(n, p_noise))

    eps = rng.normal(scale=sigma_eps, size=n)
    Y = X1 + eps

    X = np.column_stack([X1, X2, noise])
    return X, Y

# ----------------------------
# 2. Single Lasso Fit
# ----------------------------
def lasso_selected_features(X, Y, alpha=0.1):
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    model = Lasso(alpha=alpha)
    model.fit(X_scaled, Y)

    return model.coef_ != 0
```

Simulation example - Stability code

```python
# ----------------------------
# 3. Stability Selection
# ----------------------------
def stability_selection(X, Y, B=100, subsample_fraction=0.5, alpha=0.1):
    n, p = X.shape
    rng = np.random.RandomState(42)

    selection_counts = np.zeros(p)

    for b in range(B):
        idx = rng.choice(n, int(subsample_fraction * n), replace=False)
        selected = lasso_selected_features(X[idx], Y[idx], alpha=alpha)
        selection_counts += selected.astype(int)

    return selection_counts / B

# ----------------------------
# 4. Run Experiment
# ----------------------------
X, Y = generate_data(seed=1)

# Single run
single_selection = lasso_selected_features(X, Y)
print("Single Lasso selection:")
print(single_selection)

# Stability
selection_prob = stability_selection(X, Y)

feature_names = ["X1_true", "X2_correlated"] + [f"Noise_{i}" for i in range(X.shape[1]-2)]

results = pd.DataFrame({
    "Feature": feature_names,
    "Selection Probability": selection_prob
```

Simulation example - Stability selection results

```
Single Lasso selection:
[ True  True False]

Stability Selection Results:
         Feature  Selection Probability
0        X1_true                   1.00
1  X2_correlated                   0.81
2        Noise_0                   0.43
```

$p = 3, q \approx 2.2$

$$\mathbb{E}[V] \leq \frac{q^2}{(2\pi_{\text{thr}} - 1)p} \approx \frac{4.8}{(2\pi_{\text{thr}} - 1)3} \approx \frac{1.6}{2\pi_{\text{thr}} - 1}$$

For $\mathbb{E}[V] \approx 2$, $\pi_{\text{thr}} \approx 0.9$

Summary

- Stability selection a useful principle for determining a feature set
- Assigns probability to each feature
- Applies to any feature selection method